

Final Project

최종 보고서

-모바일 로봇틱스의 문제(Localization, Mapping,
Planning, Navigation) 해결 알고리즘의 시각화-

제출일: 2024.09.11

과목명: 로봇공학개론

담당교수: 최용근 교수님

전공: 컴퓨터공학과

학번: 32204045

이름: 정동재

1. 서론

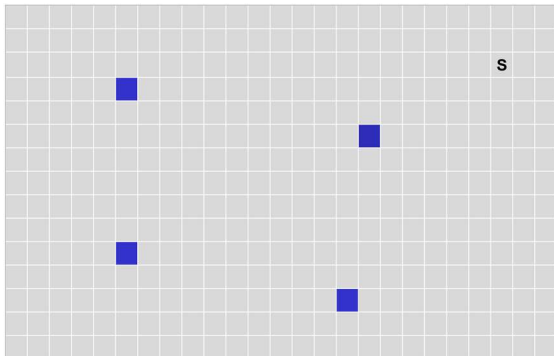
본 프로젝트는 모바일 로봇의 문제인 Localization, Mapping, Planning, Navigation의 해결 알고리즘들을 활용한다. SLAM을 활용한 Mapping과 Localization, particle filter를 활용한 Localization, A* 알고리즘과 smoothing 알고리즘을 활용한 Planning, PID Control을 활용한 Navigation을 구현한다. 이후 노이즈와 같은 주변 요소에 따라 어떤 차이가 있는지에 대해 보여준다.

2. 본론

2-1. 문제 1

문제 1에서는 SLAM을 활용하여 랜드마크들의 위치와 로봇의 위치를 추정하는 과정을 시뮬레이션 한다.

문제 1의 기본 환경은 다음과 같다.

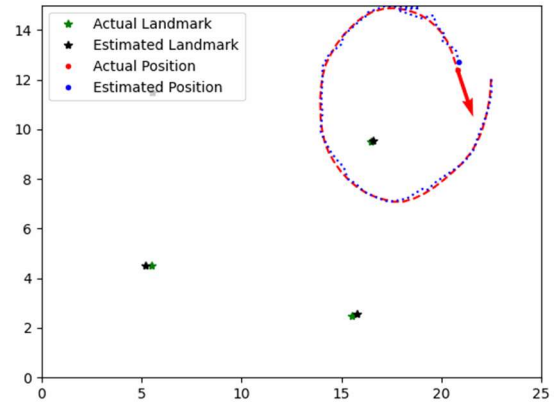


지도의 크기는 x축(가로) 25, y축(세로) 15이고 한 칸은 1m x 1m이다. 랜드마크의 위치는 (5.5, 11.5), (5.5, 4.5), (15.5, 2.5), (16.5, 9.5)이고, 로봇은 시작 pose가 S(22.5, 12.5, 1.5π)인 bicycle 모델이다. 로봇의 motion은 총 50회로 한 번 움직일 때마다 0.5미터씩 움직이고 핸들을 오른쪽으로 $2\pi/50$ 씩 움직인다. 각 랜드마크는 로봇과의 가로거리, 세로거리를 측정한다.

HANDLE_NOISE는 0.1, MEASUREMENT_NOISE는 0.2, MOTION_NOISE는 0.1로 초기 설정하였다. 모든 노이즈는 Gaussian 모델이다.

2-1-1. SLAM

로봇의 위치와 랜드마크와의 거리에 대한 관계를 행렬로 만들고 로봇이 움직일 때마다 행렬의 크기를 키워가며 업데이트 한다. SLAM 알고리즘은 수업자료로 제공받은 코드를 사용했다. 초기 에러 설정값에 대해서는 다음과 같은 결과를 얻을 수 있었다.



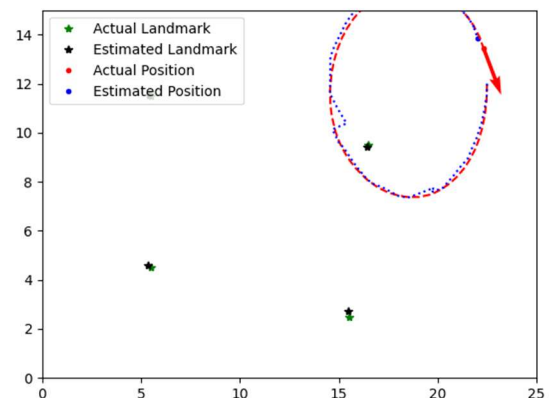
파란색 점선은 로봇의 cpu가 모터에 내린 명령을 그대로 SLAM 알고리즘에 돌려서 MOTION_NOISE와 MEASUREMENT_NOISE를 반영해서 얻은 로봇 추정 위치이다.

빨간색 점선은 로봇이 각종 NOISE의 간섭을 받는 상태에서의 실제 움직임에 대한 것이다.

이 상황에서는 SLAM 알고리즘으로 추정한 로봇의 위치와 랜드마크들의 위치가 상당히 정확한 것을 볼 수 있다.

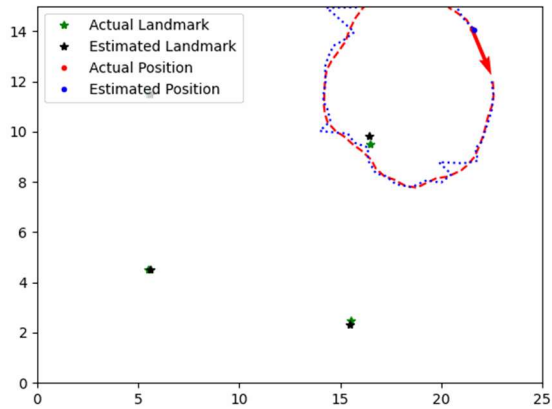
-HANDLE_NOISE 제거 또는 증가

로봇의 HANDLE_NOISE를 제거한 결과는 다음과 같다.



HANDLE_NOISE의 제거는 0.1 인 경우의 결과와 크게 다르지 않은 것으로 확인된다.

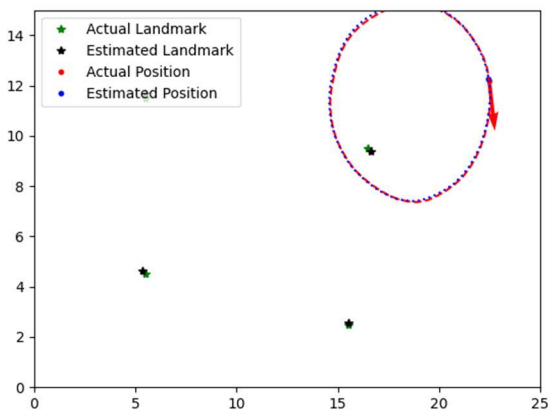
로봇의 HANDLE_NOISE를 0.5로 증가시킨 결과는 다음과 같다.



이전 결과들과는 다르게 로봇의 위치 추정에 간혹 오차가 생기고 그럴 때 랜드마크의 위치 추정도 오차가 생긴다. 그러나 결국 누적된 데이터에 의해 실제 위치에 근사하게 된다.

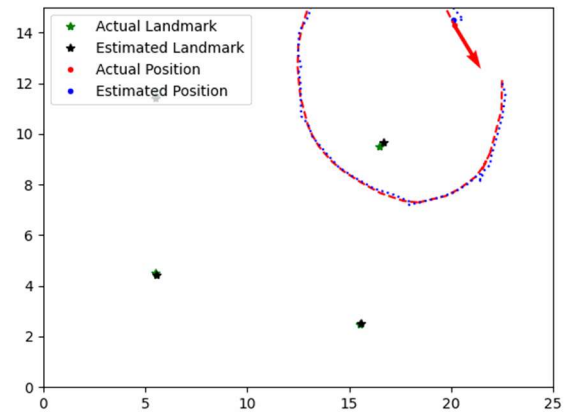
- MOTION_NOISE 제거 또는 증가

MOTION_NOISE를 0.00001로 설정한 결과는 다음과 같다.



추정 위치와 실제 로봇의 위치가 일치하는 것을 확인할 수 있다.

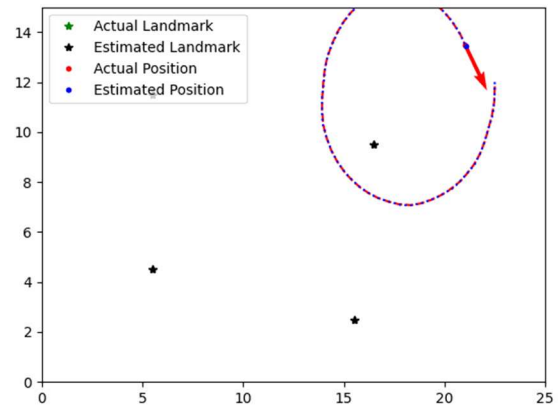
로봇의 MOTION_NOISE를 0.5로 증가시킨 결과는 다음과 같다.



이것 또한 간혹 생기는 오차를 확인할 수 있다.

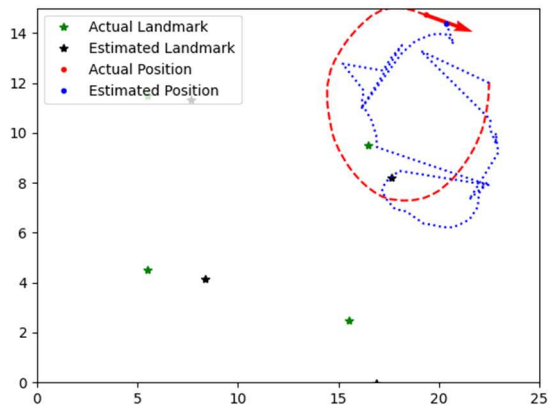
- MEASUREMENT_NOISE 제거 또는 증가

MEASUREMENT_NOISE를 0.00001로 설정한 결과는 다음과 같다.



이 경우에 작업 도중에도 랜드마크 추정 위치에 변동이 전혀 없었다. 이는 랜드마크와 로봇의 거리에 오차가 없기에 발생한 현상으로 보인다.

MEASUREMENT_NOISE를 0.5로 증가시킨 결과는 다음과 같다.



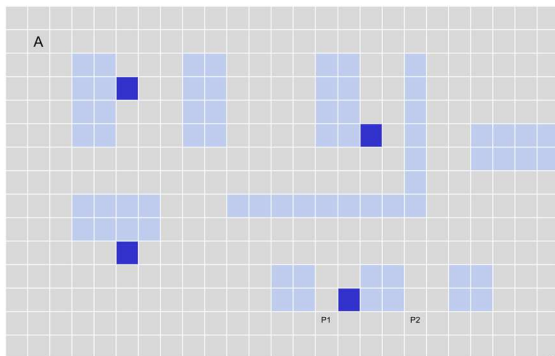
MEASUREMENT_NOISE의 증가는 다른 노이즈들의 증가와는 다르게 로봇의 위치 추정과 랜드마크 위치 추정에 큰 오차를 만드는 것을 확인할 수 있다.

이를 통해 SLAM 알고리즘에서 주변 환경을 적은 오차로 측정하는 것이 제일 중요한 것임을 알 수 있었다.

2-2. 문제 2

문제 2에서는 particle filter 알고리즘으로 로봇의 위치를 추정하고 A* 알고리즘과 Smoothing 알고리즘을 통해 목적지까지의 경로를 세워서 PID CONTROL로 로봇이 경로를 따라갈 수 있도록 하는 것이 목표이다.

문제 2의 기본 환경은 다음과 같다.



파란색 블록은 모두 장애물로 로봇이 지나갈 수 없으며, 진한 파란색은 랜드마크이다. 랜드마크는 로봇과의 거리를 최대 5m까지 측정할 수 있다. 센싱의 정확도는 Gaussian 모델로 표준편차 0.4로 설정하였다.

로봇은 bicycle 모델로 motion 모델이 완벽하지 않다는 가정하에 1m단위로 움직인다. Bicycle 모델인 만큼 steering의 최대 각도를 $(-\pi/4, \pi/4)$ 로 설정하였다. 로봇은 움직임

마다 해당 각도 범위 내에서 무작위로 1미터를 움직인다. 로봇이 장애물에 부딪히면 각도를 조금씩 틀어서 부딪히지 않는 위치로 이동할 수 있도록 하였다.

문제 2에서는 P1을 목적지로 설정한다.

Steering_noise, distance_noise는 각각 gaussian 모델로 표준편차 0.1, 0.1로 설정하였다.

2-2-1. Localization

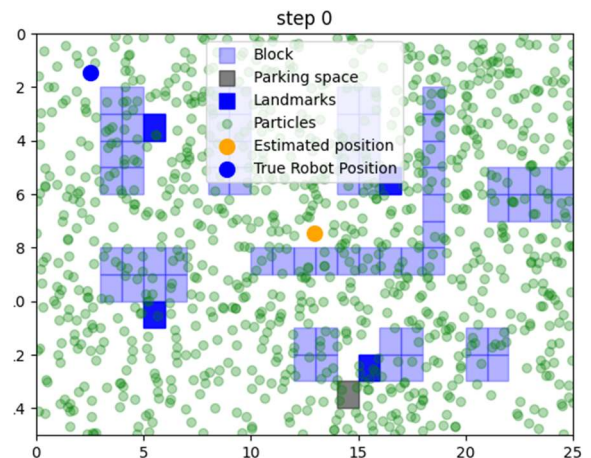
로봇의 출발점을 A라고 가정했을 때 Particle filter 알고리즘을 활용하여 로봇의 현재 위치를 추정한다.

샘플의 개수는 1000개로 설정하였고, Gaussian 확률 분포 함수를 사용하여 가중치를 계산하였다. 또한 샘플들이 한 점에 모이는 것을 방지해서 각 샘플의 move에 노이즈를 적용하였다.

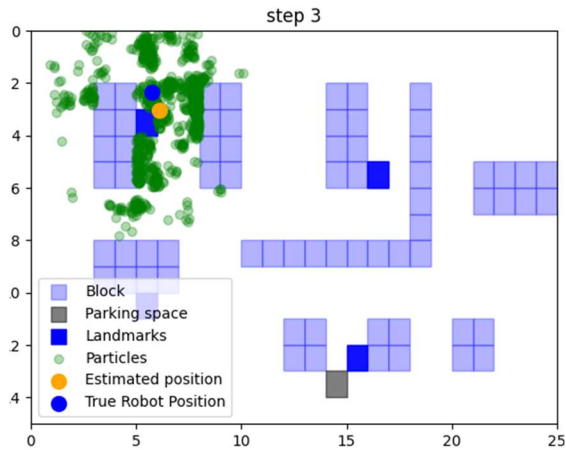
샘플들 위치의 평균을 추정 위치로 판단한다.

맵은 좌측 상단이 (0,0)으로 우측 하단으로 갈수록 x, y값이 커진다. 각도는 좌측을 향할 때 0, 시계 방향으로 커진다.

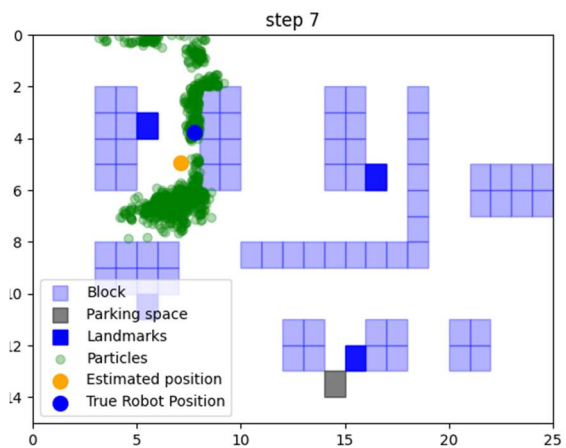
Particle filter로 로봇의 위치를 추정하기 위해 무작위 move를 반복해 실행한 결과는 다음과 같다.



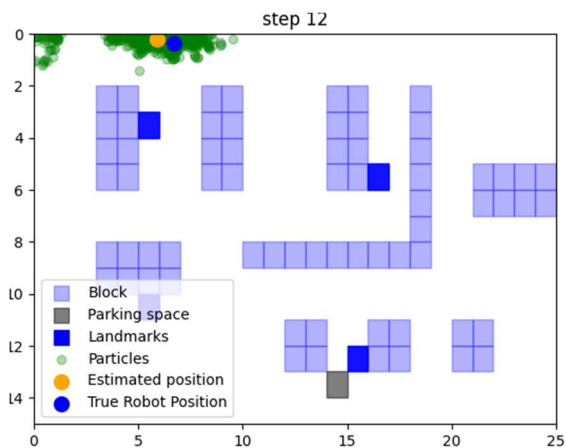
센싱을 하기 전에는 맵 전체에 샘플들이 퍼져 있는 것 확인할 수 있다.



적은 횟수의 반복으로는 제한된 정보만을 측정하기에 샘플들이 조금은 퍼져 있는 것을 확인할 수 있다. 이때의 오차는 2.254이다.



측정의 최대값이 5m를 넘지 않고, 랜드마크 사이에 모든 랜드마크와의 거리가 5m가 넘는 위치가 많이 존재하기에 로봇이 어떤 움직임을 가지느냐에 따라 다시 오차가 커지는 것을 확인할 수 있다. 이때의 오차는 2.635이다

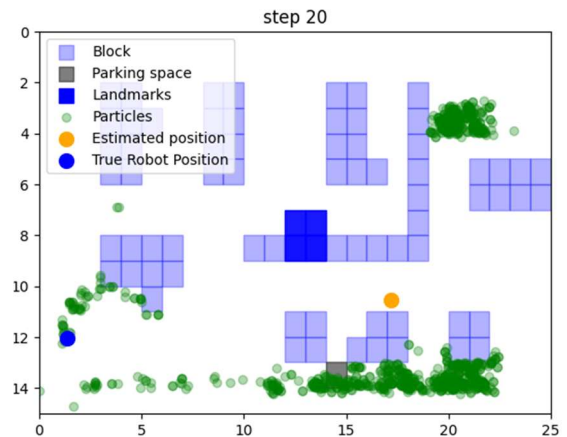


이번 실행에서는 12번의 움직임이 다 끝난

뒤 오차 0.899로 높은 정확도의 위치 추정을 할 수 있었다. 이 횟수는 로봇의 랜덤한 움직임이 어떻게 되는지에 따라 달라진다. 샘플들의 평균 위치는 (6.058384, 0.211920)이고, 실제 로봇 위치는 (6.720637, 0.368539)으로 정확도가 굉장히 높은 것을 확인할 수 있다.

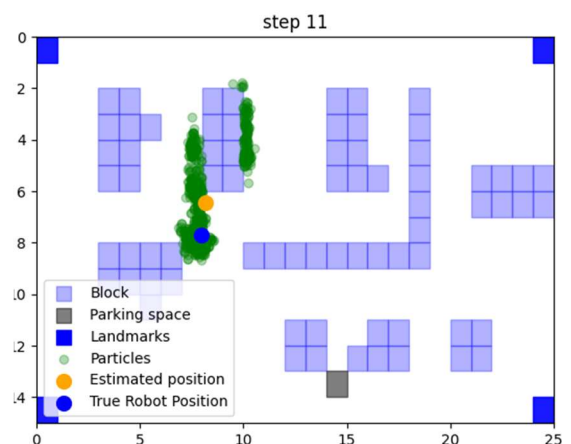
-랜드마크 위치와의 연관성.

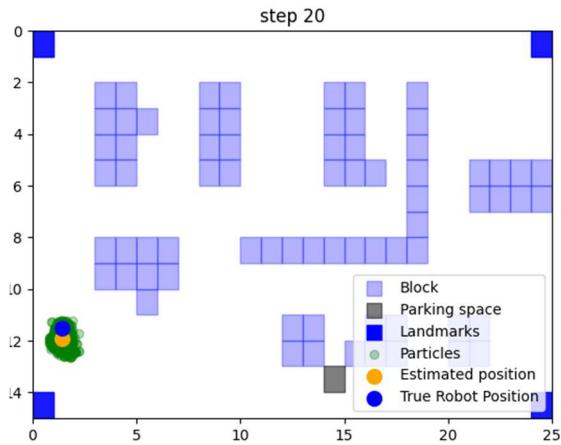
랜드마크의 위치를 더 밀집하게 만들어서 실행한 결과는 다음과 같다.



Particle filter를 20회 돌렸는데도 샘플들이 맵 전체에 퍼져있는 것을 확인할 수 있다. 실제 로봇의 위치는 (1.351, 12.055)인 반면, 추정 위치는 (17.216, 10.540)이다. 오차는 16.5로 매우 크다.

랜드마크의 위치를 각 모서리에 두고 실행한 결과는 다음과 같다.

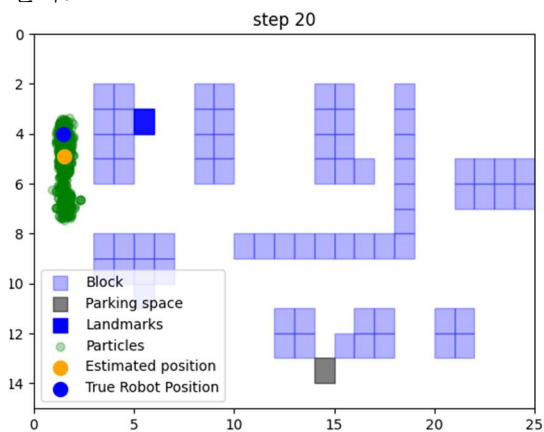




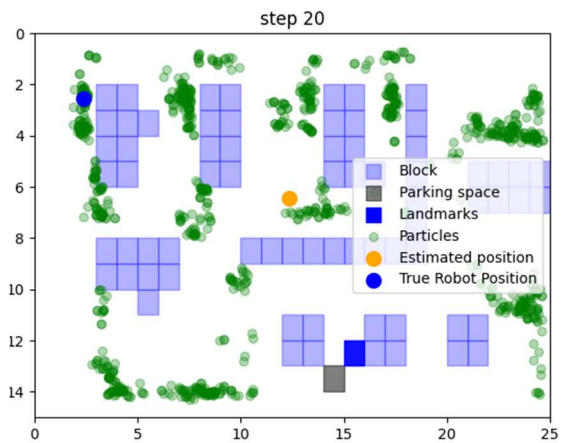
이 때의 정확도는 로봇이 맵의 중앙에 있을 때 낮아지고 외각으로 빠지면 정확도가 올라간다는 것을 확인할 수 있다.

-랜드마크 숫자와의 연관성.

랜드마크의 수를 1개로 줄인 결과는 다음과 같다.



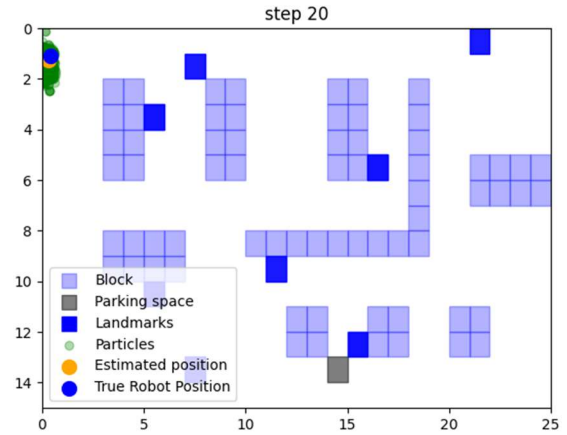
로봇의 위치가 랜드마크의 최대 측정거리보다 가까이 있으면 꽤 높은 정확도의 추정 위치를 구할 수 있다는 것을 확인할 수 있다.



랜드마크의 측정 가능 거리 바깥에 로봇이

위치한다면 로봇의 위치를 추정하기에 어려움이 많다는 것을 확인할 수 있다.

다음은 랜드마크의 수를 8개로 늘리고 실행하였다.

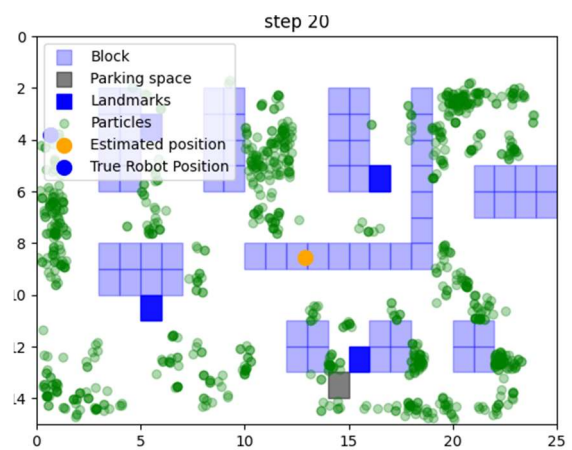


이 때의 오차는 0.36으로 랜드마크가 4개였을 때보다 높은 정확도를 확인할 수 있다.

이를 통해 랜드마크의 수가 많을수록 로봇의 위치를 더 정확하게 추정할 수 있다는 것을 알 수 있다.

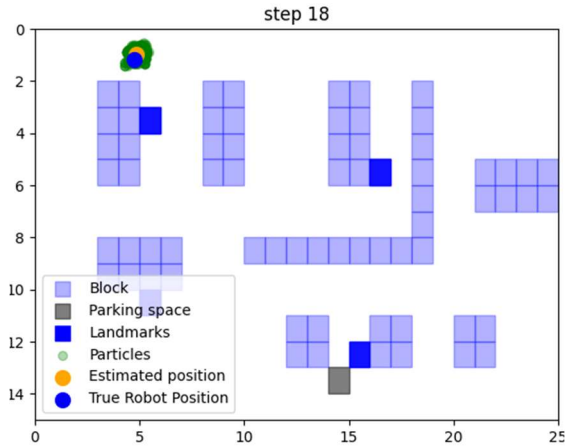
-랜드마크의 최대 측정 거리와의 연관성.

랜드마크의 최대 측정 거리가 1m일 때의 결과는 다음과 같다.



로봇의 위치를 추정하지 못할 정도로 샘플들이 퍼져 있는 것을 확인할 수 있다.

랜드마크의 최대 측정 거리를 20으로 늘리고 실행한 결과는 다음과 같다.



이 때의 오차는 0.25로 최대거리가 5일때보다 빠르고 정확하게 로봇의 위치를 추정할 수 있다는 것을 알 수 있다.

각 조건에 대한 정확도는 다음과 같이 정리할 수 있다.

	감소(밀집)	증가(분산)
랜드마크 위치	감소	증가 (조건부)
랜드마크 개수	감소(조건부)	증가
최대 측정 거리	감소	증가

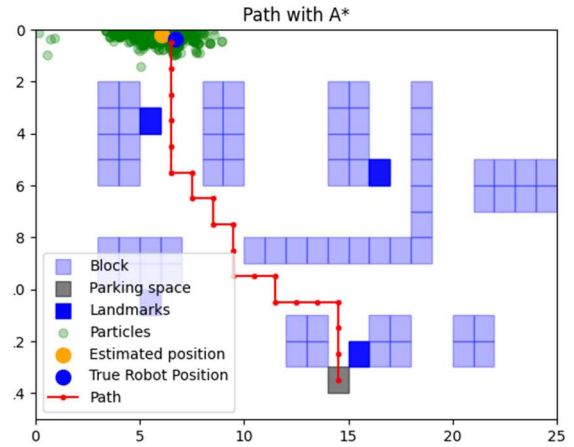
2-3. 문제 3

Particle filter 알고리즘을 이용한 Localization이 완료됐으면 이제 목적지인 P1을 향한 경로를 찾아야한다. 이를 위해서 A* 알고리즘을 활용하여 최적경로를 구한다. 이후 Smoothing 알고리즘을 활용해 로봇의 움직임에 더 알맞은 경로로 바꿔준다.

이후에 PID Control 알고리즘을 활용하여 로봇에 적절한 이동 명령을 내려서 목적지에 도달할 수 있도록 한다.

2-3-1. Planning

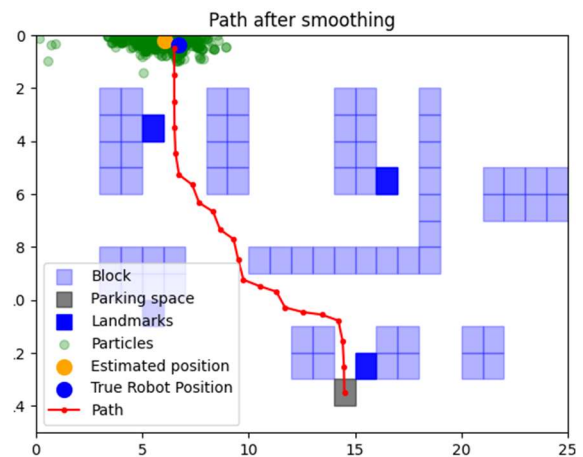
로봇의 추정 위치인 (6.058384, 0.211920)가 있는 칸의 중앙값인 (6.5, 0.5)에서부터 목적지의 중앙값인 (13.5, 14.5)까지의 최적경로는 다음과 같다.



A*알고리즘을 사용해서 구한 경로이므로 이 경로는 최단경로이다.

이 경로는 bicycle 모델인 로봇의 움직임으로 따라가기에는 많은 한계가 있기 때문에 Smoothing 알고리즘을 활용하여 경로를 로봇이 갈 수 있도록 바꿔준다.

Smoothing된 경로는 다음과 같다.



Smoothing의 기존 경로에 근접하도록 하는 가중치를 0.8, 좀 더 부드러워지도록 하는 가중치를 0.4로 설정하였다. 이는 너무 부드럽게 만들면 이동 중에 장애물에 부딪히는데 이때의 대처 방식이 방향을 틀도록 설정하여서 이상한 곳으로 가는 것을 방지하기 위함이다.

2-3-2. Navigation.

경로를 구했다면 이제 로봇에게 이동 명령을 내릴 차례이다. PID Control 알고리즘을 활용한다.

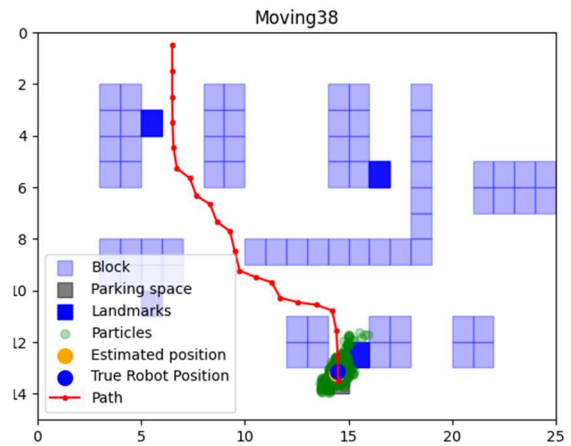
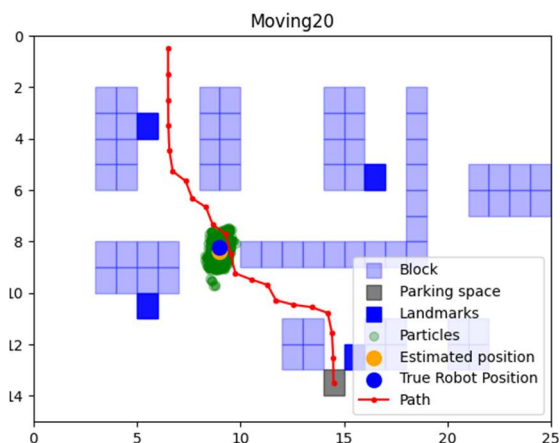
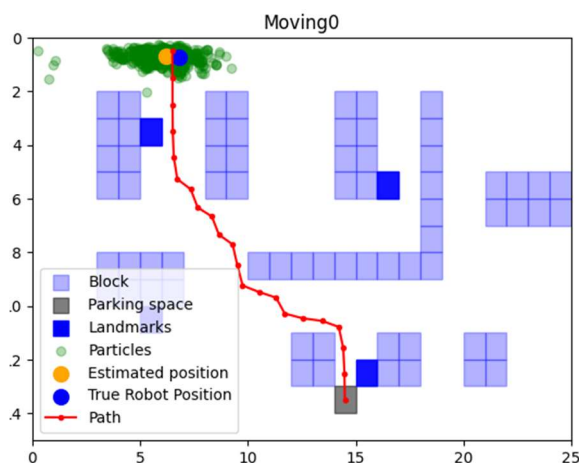
Set point와 process variable의 차인 에러

를 이 문제에서는 로봇의 추정 위치와 경로와의 가장 가까운 수직 거리로 설정하였다. 외적을 이용하여 계산하면 로봇이 현재 경로의 왼쪽에 있는지 오른쪽에 있는지도 에러값에 부호로 반영할 수 있다. 에러가 음수인 경우 로봇이 진행 경로의 좌측에 있는 것이고, 양수이면 우측에 있는 것이다. 만약 좌측에 있다면 핸들을 오른쪽으로 꺾도록 하고, 우측에 대해서는 반대로 한다.

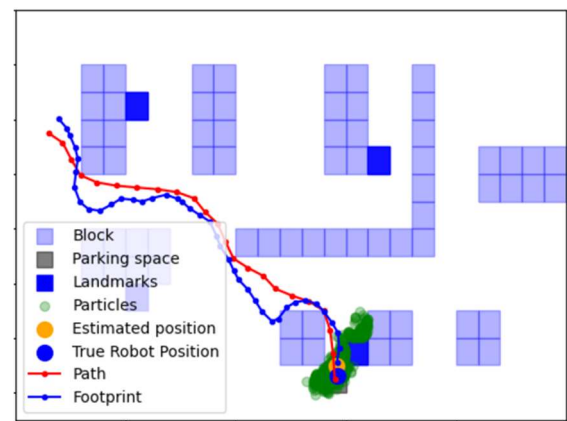
이 에러에 대한 Control signal인 $P+I+D$ 를 계산한 후 여기에 로봇의 진행방향과 경로의 진행방향 각도의 차이를 반영하여 로봇의 move를 위한 steering으로 설정한 뒤, 0.5m 씩 이동을 수행하게 하였다. 0.5m 이동에 대한 이유는 움직임을 더욱 세심하게 하기 위해서이다.

로봇이 이동할 때마다 샘플들에도 움직임을 적용하여 계속해서 localization을 수행할 수 있도록 하였다.

이동 과정은 다음과 같다.



총 39번의 움직임으로 목적지인 P1까지 도달할 수 있었다.



위 그림은 다른 실행에서의 smoothing 경로와 실제 로봇의 움직임을 출력한 화면이다.

2-4. 문제 4

지도와 상황은 아까와 같지만 이번에는 랜드마크와의 거리를 측정하는 것이 아니라, 로봇이 바라보는 방향 기준으로 주변 360도를 10도단위로 끊어서 장애물을 측정하는 상황이다. 최대 거리 8m 까지 측정 가능하며 센서의 정확도는 표준편차 0.2m인 Gaussian 모델을 사용한다.

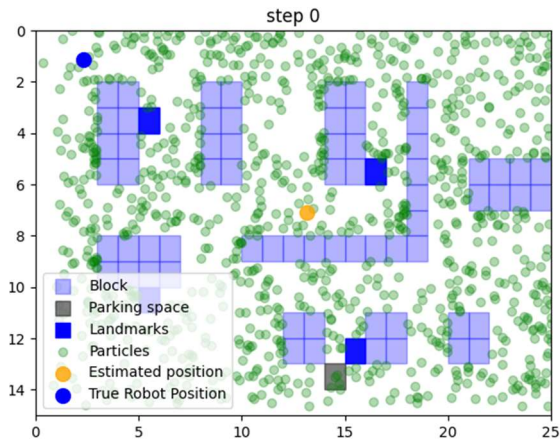
Steering_noise, distance_noise는 각각 gaussian 모델로 표준편차 0.1, 0.1로 설정하였다.

출발점을 A라고 가정했을 때 자신의 위치를 추정해야한다.

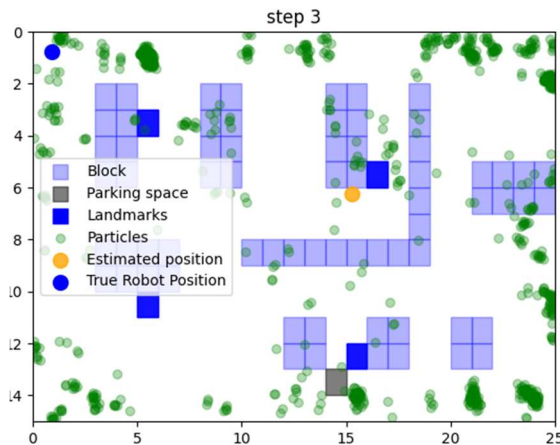
2-4-1. Localization

문제 4에서도 문제 2와 같이 particle filter를 사용하여 로봇의 위치를 추정한다. 달라진 점은 샘플들의 가중치를 랜드마크들과의 거리가 아닌 주변 360도에서의 장애물과의 거리에 대해 설정한다는 것이다.

360도를 10도씩 측정하고, 측정거리를 0.2m씩 늘려가면서 장애물이 발견되면 해당 거리를 저장하는 방식으로 구현하였다.



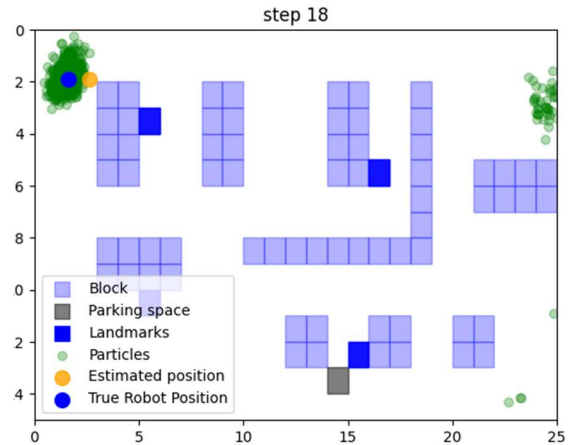
처음에 파티클들의 orientation을 랜덤으로 설정하여 최대한 현재 로봇의 pose와 유사한 파티클이 존재할 수 있도록 하였다.



로봇이 무작위로 움직이며 얻는 센싱 정보와 유사한 위치에 파티클들이 몰리는 것을 확인할 수 있다. 이 때의 오차는 16.542로 매우 큰 것을 확인할 수 있다.

파티클을 로봇의 센싱 정보와 비교해서 가중치를 낼 때 사용하는 Gaussian 메소드의 sigma의 값을 7.5로 크게 설정하여 초반에 파티클들이 특정 위치에 몰려서 잘못된 위치에 근사하게 되는 경우를 방지하였다. 이로 인해 처음에 잘못된 파티클들의 가중치가 올라가더라도 다시 로봇의 진짜 위치를 추정할

수 있다.

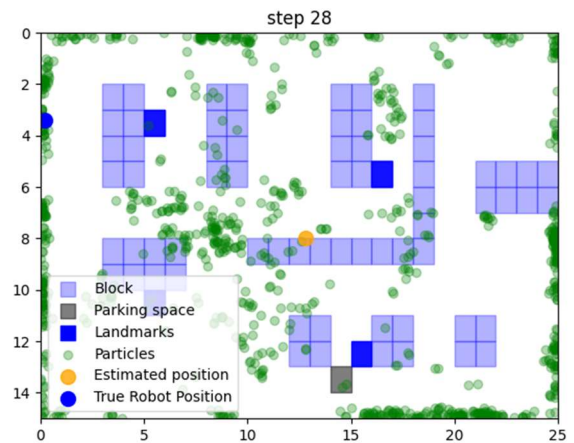


19회의 무작위한 움직임을 하면서 쌓인 주변 장애물에 대한 데이터로 로봇의 위치를 꽤 정확하게 추정할 수 있었다. 이때의 실제 로봇의 위치는 (1.630380, 1.917465)이고, 추정 위치는 (1.876426, 1.831744)이며 오차는 0.664이다.

문제 2와의 측정에 비해 가중치를 계산할 때 비교해야하는 값들이 많아서 실행시간이 더 오래 걸리는 것을 확인할 수 있었다.

-최대 반경 감소 증가

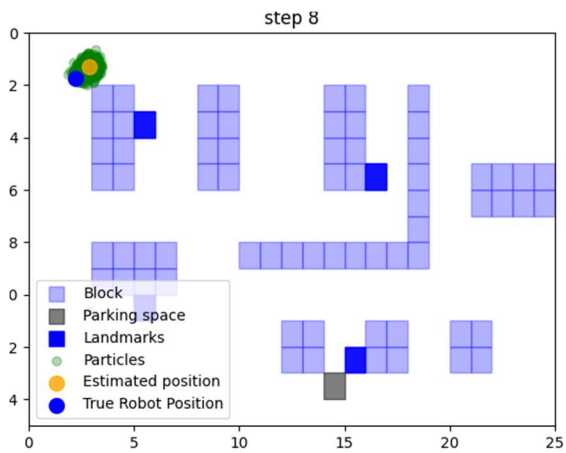
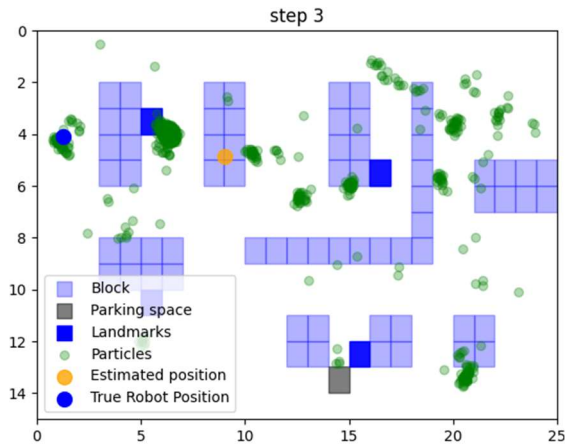
측정 가능 최대 반경을 1m로 변경한 결과는 다음과 같다.



짧은 거리밖에 측정 못하기에 다른 위치에서 비슷한 측정값이 나올 확률이 증가하여 28번의 particle filter를 수행했는데도 파티클들이 전혀 밀집되지 않아 로봇의 위치를 특정할 수 없는 것을 확인할 수 있다.

측정 가능 최대 반경을 16m로 변경한 결과

는 다음과 같다.

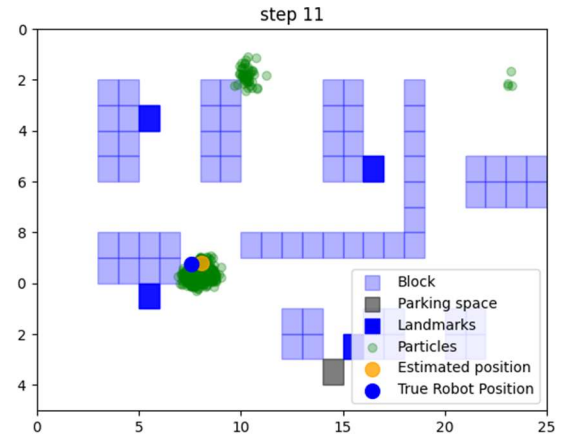


측정 반경이 8m였을 때보다 훨씬 빠른 속도로 오차 0.706의 정확한 위치를 추정할 수 있었다.

이를 통해 최대 반경이 클수록 로봇의 위치를 빠르고 정확하게 추정할 수 있다는 것을 확인할 수 있다.

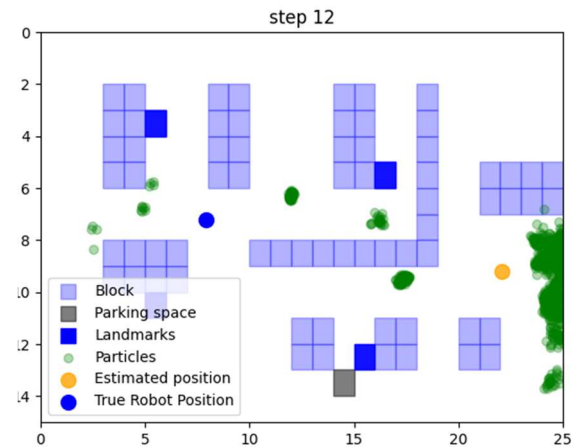
-측정 정확도 감소 증가

측정 정확도의 표준편차를 0.001로 감소시킨 결과는 다음과 같다.



11회 반복으로 오차 0.540의 추정 위치를 구하였다.

표준편차를 3으로 증가시킨 결과는 다음과 같다.



로봇이 다양한 주변 센싱 데이터를 얻을 수 있는 움직임을 가져왔지만, 이전 결과와 달리 로봇의 위치에 근접하지도 않은 것을 확인할 수 있다.

이를 통해 센서의 정확도가 떨어질수록 로봇 추정 위치의 정확도도 감소한다는 것을 알 수 있다.

각 조건에 대한 정확도는 다음과 같이 정리할 수 있다.

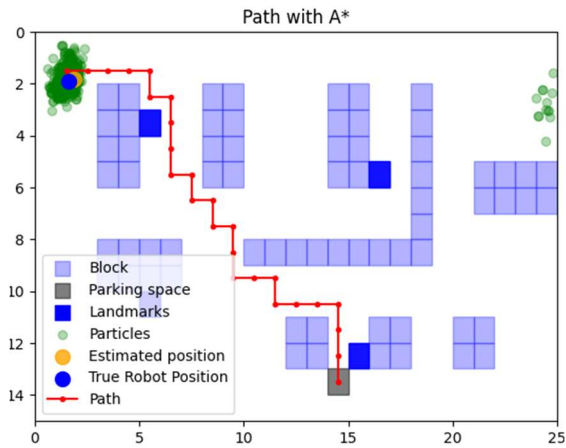
	감소(밀집)	증가(분산)
측정 반경	감소	증가
측정 정확도	감소	증가

2-5. Planning & Navigation

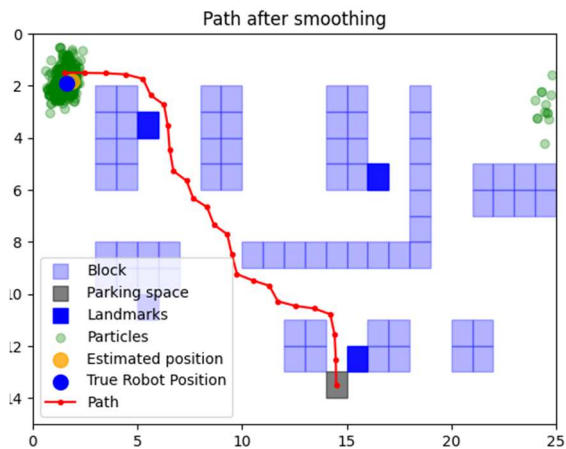
Planning 과 Navigation 과정은 이전 문제 3의 방식과 동일하다.

2-5-1. Planning

A*알고리즘을 활용해 최적경로를 얻는다.

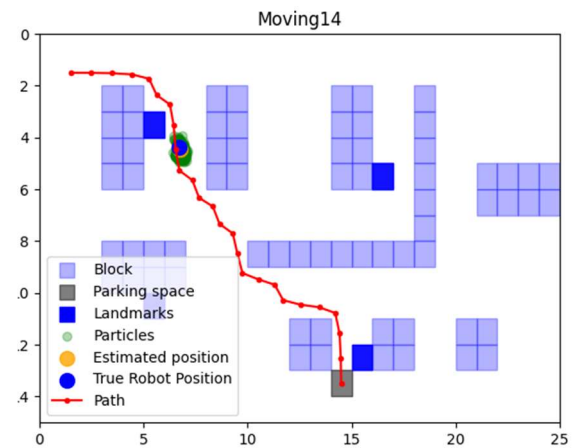
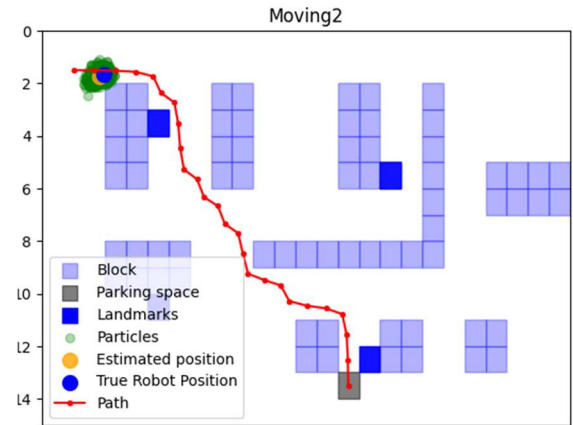


Smoothing 알고리즘으로 경로를 부드럽게 만들어준다.

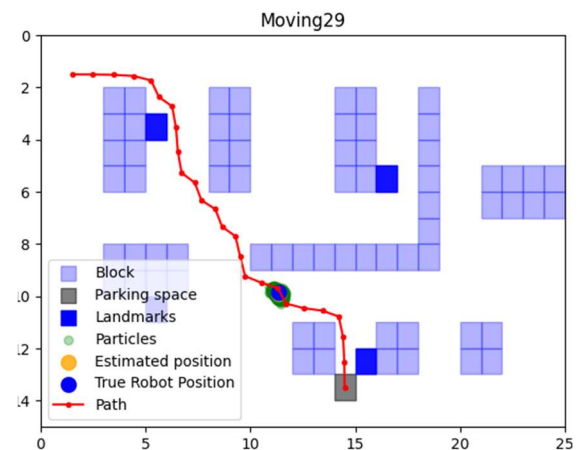


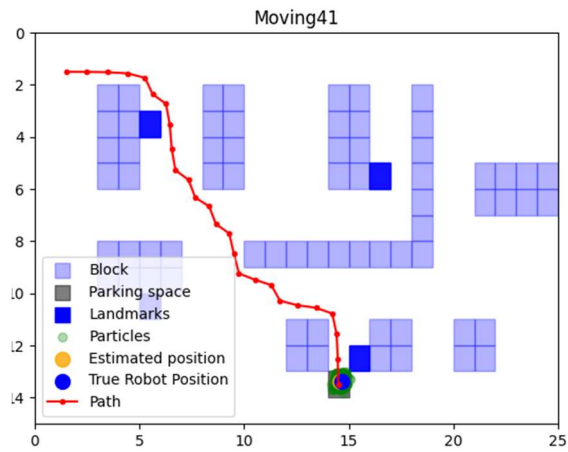
2-5-2. Navigation

PID Control 로직도 문제3에서와 동일하다.

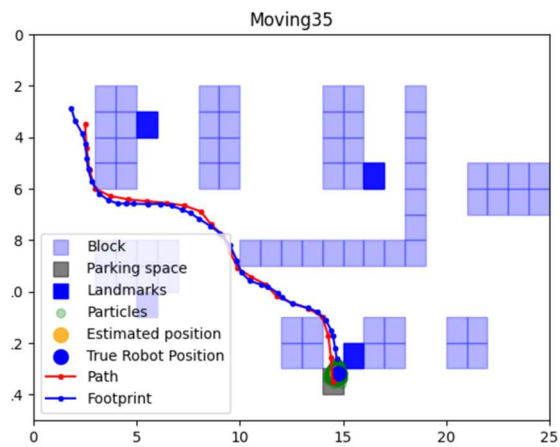


경로를 따라 이동하면서 얻는 주변 장애물에 대한 측정 데이터로 파티클의 밀집도가 올라가고, 추정 위치와 실제 위치의 오차가 거의 없는 것을 확인할 수 있다.





경로를 따라 P1에 주차한 것을 확인할 수 있다.



같은 실행 상황은 아니지만 도착지에 가는 도중의 실제 로봇의 움직임(footprint)을 출력한 결과이다.

3. 결론

SLAM, Particle filter, A*, smoothing, PID 알고리즘을 사용하여 모바일 로봇의 전반적인 동작 원리를 구현하는 프로젝트를 진행하였다. 각각의 노이즈나 환경 상황에 따른 측정값의 변화나 오차 등에 대해서 학습할 수 있었다.